RETRACE: MODEL-RESPONSE WATERMARKING FOR DIFFUSION LANGUAGE MODELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Existing language-model watermarks are detected from token patterns. We detect ours from *how a diffusion language model responds when the generated text is partially masked and reconstructed*. Because such a model is trained to recover masked tokens from arbitrary surviving context, a finished text can be pushed back through the operator that produced it: corrupted with a secret key, re-denoised, and its response read. ReTrace places a watermark in that response. A keyed pair of context ablations induces a per-position log-likelihood contrast, and the watermark asks the sign of that contrast to agree with a keyed orientation, giving a test statistic that is a sum of bounded indicators with a finite-sample exact Binomial$(n, \frac{1}{2})$ null. Embedding never overrides the model: decoding follows the reference block schedule, and the watermark only decides whether a sampled proposal is emitted or redrawn. We also report the negative result that motivated this design — post-hoc token substitution is priced out on a well-trained diffusion language model, and, because both are governed by the sharpness of the same conditionals, *improving generation quality reduces post-hoc watermarkability*.

1 INTRODUCTION

Existing language-model watermarks encode provenance in *what tokens are produced*. Red–green schemes bias the token distribution toward a keyed vocabulary partition (Kirchenbauer et al., 2023); distortion-free schemes fix the sampling randomness with a keyed pseudorandom sequence; and the watermarks designed for diffusion language models exploit the extra freedom those models expose — unresolved context, per-step sampling randomness, global sequence statistics, or the order in which positions are unmasked (Hong & No, 2026). Different as they are, every one of them leaves the verifier reading a *surface signature*: a hash of token identities, a frequency, a rank. We ask a different question.

Can a watermark be detected from how the model reconstructs a text, rather than from the tokens the text contains?

A diffusion language model makes this question answerable in a way an autoregressive model does not. Its training objective is to recover masked tokens from arbitrary surviving context, so any finished text can be pushed *back* through the operator that produced it: corrupt it, re-denoise it, and observe the response. That response is a property of the text's relationship to the model, not of its symbols, and it is available to anyone holding the model — but only interpretable by someone holding the key that chose the corruption.

We call the resulting scheme **ReTrace**. A secret key derives two corruption patterns over the context of a span. Reading the model's log-probability of each *observed* token under both corruptions gives a contrast

$$g_i(y_i) \;=\; \log p_\theta\big(y_i \mid C_i^1(y)\big) - \log p_\theta\big(y_i \mid C_i^0(y)\big)\,, \tag{1}$$

and the watermark asks not that y_i belong to a keyed vocabulary subset, but that the sign of $g_i(y_i)$ agree with a keyed orientation. The evidence lives in the model's behaviour.

Getting the embedding right took one full negative result (§5). Substituting tokens — the obvious embedding — fails because a well-trained denoiser's conditionals are sharp: the median position admits exactly one token inside any usable fluency budget, and the effect *strengthens* as generation

improves, a sampler repair that lowered draft perplexity from 22.2 to 9.5 also shrinking the admissible set by 34–60 %. *Better diffusion-model generation reduces post-hoc watermarkability*, and it rules out an entire family of designs.

What survives is to never leave the model's own conditional. At a keyed carrier the token is drawn from p_θ and simply *redrawn* — at most R times, at no forward-pass cost, the conditional row being unchanged — until its reconstruction response agrees with the key; if no draw agrees, one unconditional draw is emitted as-is. Positions whose plausible tokens all answer one way fall through to the natural sample either way, so the watermark is paid for only at genuinely ambivalent positions. *ReTrace places the watermark in resampled tokens and reads it back from the model's reconstruction response* (Figure **??**).

In sum: a watermark read from the model's response rather than its tokens, verifiable from text, model and key alone (§4); a counting statistic with a finite-sample exact binomial null under the ideal-PRF model, false-positive control verified at the *worst* of twenty keys (§4.2); an embedding that never leaves the model's conditional (§4.3); and a mechanism-level negative result — substitution is priced out, and better generation makes it worse (§5).

2 Related Work

Diffusion language models. Continuous diffusion (Ho et al., 2020) was carried to discrete data by structured corruption processes (Austin et al., 2021) and by score-ratio and masked-diffusion formulations (Lou et al., 2024; Sahoo et al., 2024; Shi et al., 2024; Ou et al., 2025). Instruction-following models at language-model scale followed — LLaDA and its successors (Nie et al., 2025; Zhu et al., 2025), Dream (Ye et al., 2025) — alongside industrial systems built on the same paradigm (Labs et al., 2025). The property we exploit is the training objective itself: these models predict masked tokens from arbitrary surviving context, so they can be applied to a *finished* text and not only used to produce one.

Two further findings from this literature shape our design. First, practical dLLMs are *not* order-invariant even though an ideal conditional predictor would be, and the choice of unmasking order measurably changes what is generated (Kim et al., 2025). Second, decoding is in practice semi-autoregressive: blocks are filled in sequence and diffusion happens within a block (Arriola et al., 2025), and work on accelerating that loop shows that committing a confident token whose left context is still unresolved damages coherence (Wu et al., 2026; Wei et al., 2026; Ben-Hamu et al., 2025). Our embedder inherits both: it steers only *within* the reference block schedule, and it borrows the frontier-anchored commit rule rather than inventing a schedule of its own.

Watermarks for autoregressive models. The dominant family biases token probabilities. Kirchenbauer et al. (2023) seed a green/red vocabulary partition on the preceding token and add a logit bias; detection counts green tokens against a binomial null, and the scheme's reliability and robustness have been studied extensively (Kirchenbauer et al., 2024; Zhao et al., 2023; 2024). A second family avoids distortion by fixing the sampling randomness with a keyed pseudorandom sequence, either through the Gumbel-max trick (Aaronson & Kirchner, 2023; Fu et al., 2024) or with explicit distortion-freeness and undetectability guarantees (Kuditipudi et al., 2024; Christ et al., 2024); others place the signature in model parameters rather than in sampling (Block et al., 2025). Surveys cover the space (Petitcolas et al., 1999; Liang et al., 2026), and the threat model is correspondingly well studied, from paraphrase attacks (Krishna et al., 2023) to extraction of the key itself (Jovanović et al., 2024). What every one of these has in common is the *verification* step: the detector computes a function of the token identities.

Watermarks beyond left-to-right generation. Because the green list is seeded by a prefix that order-agnostic decoding does not provide, transferring the autoregressive constructions to dLLMs requires care. Chen et al. (2025) give a watermark for order-agnostic models by biasing token selection in a way that does not assume a prefix. For diffusion language models specifically, Gloaguen et al. (2026) apply a red–green bias *in expectation* over unresolved context and additionally prefer tokens that make other positions green; Bagchi et al. (2025) use keyed Gumbel-max sampling at each denoising step while preserving the sampling distribution; Wu et al. (2025) and Raban et al. (2026) target the same setting with order-agnostic and efficiency-oriented constructions. Closest

to our embedding channel is dgMARK (Hong & No, 2026), which leaves the learned conditionals untouched and instead prefers to unmask positions whose already-preferred token satisfies a keyed parity condition; that discipline is precisely why it is inexpensive, and we adopt it.

What is new here. All of the above differ in *where the watermark is placed* — token probabilities, sampling randomness, decoding order — while agreeing on *what the verifier reads*: a function of the emitted symbols. ReTrace changes the second. The verifier masks part of the finished text under the key, has the model reconstruct it, and compares the reconstructions; the evidence is the model's response, which no token-level statistic can express. The price is equally clear and we report it in every table: those detectors need no model calls, ours needs a fixed number of sequence evaluations. To our knowledge no prior watermark for language models — autoregressive or diffusion — uses the generator itself as the verification oracle in this way.

3 Background and Problem Setup

DLM generation. A diffusion language model reverses a masking corruption: inference fills a masked region over T steps, committing the most confident positions each step, and practical systems decode semi-autoregressively — contiguous blocks left to right, diffusion within the active block (Arriola et al., 2025; Nie et al., 2025). Two consequences matter. The model predicts masked tokens from *arbitrary* surviving context, so it can be applied to a finished text and not only used to produce one; and each emitted token is a sample from a conditional the verifier can reconstruct exactly, since that conditional sees only the finished prefix and masks.

Watermarking problem. An embedder holding a secret key K produces text whose provenance a verifier holding the same key can later establish from the text alone, against three requirements. *Detectability:* a test statistic separating watermarked from natural text at a controlled false positive rate. *Quality:* the watermarked distribution should cost little against the same generator without the key. *Verifiability without the trajectory:* detection may use the text, the model and the key, but nothing recorded at generation time. Prior schemes meet these by making the verifier read a function of the token identities — a green-list membership seeded by the preceding token (Kirchenbauer et al., 2023), a parity of the unmasking order (Hong & No, 2026), a keyed sampling sequence (Kuditipudi et al., 2024). This paper asks the verifier to read something else: how the model *reconstructs* the text under keyed corruption. The cost of that choice is a detector that must run the model, and we account for it in every comparison.

4 ReTrace

4.1 The observable

Let y be a generated span and K a secret key. For a block B of y, the key derives L *ablation patterns* $D_1, \ldots, D_L$ over the context preceding B, and a pair (u, v) among them. Writing $\mathscr{C}_\ell$ for the context with B, everything after B, and D_ℓ masked, the *challenge contrast* at position $i \in B$ is

$$g_i(w) \;=\; \log p_\theta(w \mid \mathscr{C}_v) - \log p_\theta(w \mid \mathscr{C}_u)\,, \qquad w \in \mathscr{V}. \tag{2}$$

Two properties of $\mathscr{C}_\ell$ matter and neither is optional.

The block and its future are masked in every arm. While B is being decoded, every later block is still `[MASK]`. If the verifier recomputed Eq. (2) on the finished text without re-masking that tail, it would condition on tokens that did not exist when the guidance was produced, and the two sides would be reading different functions. Masking B and everything after it makes the table identical at both times; masking B itself additionally makes it *invariant* to whatever is committed inside B, so a single table serves the block's entire decoding.

The ablations are drawn from the already-final region. $D_\ell \subset \text{prompt} \cup B_{<b}$, which the verifier has verbatim. We use a *contrast* pair — one pattern ablating the context nearest the block, the other the furthest — because recency dominates next-token prediction, so the two arms disagree far more than two random ablations do, widening the dynamic range of g.

4.2 THE STATISTIC

Each position carries an independent keyed orientation bit $\varepsilon_i \in \{\pm 1\}$ and a target $w_i \in \{\pm 1\}$, and contributes a bounded indicator

$$m_i \;=\; \mathbf{1}[w_i\,\varepsilon_i\,g_i(y_i) > 0]\,, \qquad T \;=\; \sum_{i\in P} m_i. \tag{3}$$

Proposition 1 (Exact null)**.** *Let the challenge patterns, the per-position pair selection, the carrier set, the group assignment, the tie coins and the orientation bits each be derived from the key under* separate PRF domains. *Fix a text y independent of the key, and condition on the carrier set, the selected pairs, and the challenge magnitudes — all of which are functions of y and the non-orientation domains only. Under the ideal-PRF model the orientation bits are then independent Rademacher variables independent of everything conditioned on, so the m_i are i.i.d.* Bernoulli$(\frac{1}{2})$ *and* $T \sim$ Binomial$(|P|, \frac{1}{2})$ *exactly.*

The domain separation is load-bearing: the challenge patterns and pair selection are themselves keyed, and sharing PRF output with the orientation bits would let the conditioning event carry information about them. Pair selection uses only the swap-invariant S_i of Eq. (7), so it is orientation-symmetric by construction.

The proposition needs no calibration corpus and no model-specific null estimate, and it is finite-sample rather than asymptotic. Three implementation details are load-bearing.

Ties must be resolved, not dropped. In single precision the emitted token's log-probability rounds to 0 under both arms at a quarter of positions, and scoring the vanished difference as a miss dragged the unwatermarked match rate to 0.32–0.37 against 0.50. Double precision inside the softmax removes the ties; a second keyed coin breaks any residue.

Counting, not averaging. The per-position contrast is heavy-tailed, so an average is dominated by its largest values and sign control has no leverage on it; a count gives every steered position exactly one unit.

Presence, verification and payload are separate tests. Summing hits across payload groups cancels — a balanced message drives half toward $m_i{=}1$ and half toward $m_i{=}0$, so a perfect watermark sums to $|P|/2$. A presence pool with $w_i \equiv +1$ tests provenance with no claimed message (and never tests a message decoded from the same data); claimed-message verification aligns each group to its bit; payload thresholds each group separately.

4.3 EMBEDDING BY RESPONSE-GUIDED RESAMPLING

Generation follows the reference schedule of the base model — blocks in order, diffusion within a block — and commits positions in plain confidence order (Figure 1). At a carrier position the committed token is chosen by rejection sampling from the model's own conditional:

$$v^{(1)}, v^{(2)}, \ldots \sim p_\theta(\cdot \mid x), \qquad \text{emit the first } v^{(r)} \text{ with } w_i\,\varepsilon_i\,g_i(v^{(r)}) > 0, \quad r \le R, \tag{4}$$

and if all R guided draws are rejected, one further unconditional draw is emitted as-is. A retry costs no forward pass — the conditional row is unchanged — so R trades detection strength against arithmetic alone. Writing A_i for the acceptance set and q_i for its mass, the per-position hit probability and emitted distribution are

$$q_i = \Pr_{v\sim p_i}[w_i\,\varepsilon_i\,g_i(v) > 0]\,, \qquad \Pr[m_i = 1] \;=\; 1-(1-q_i)^{R+1}, \tag{5}$$

$$v_i \;\sim\; \big(1-(1-q_i)^R\big)\,p_i(\cdot \mid A_i) \;+\; (1-q_i)^R\,p_i. \tag{6}$$

This *does* condition the sampling distribution on the acceptance predicate and is not claimed to be distortion-free; what makes it cheap is that the support never leaves the model's own conditional, and that the positions where the predicate binds hardest are exactly the positions where it cannot be satisfied at all — which fall through to the natural sample.

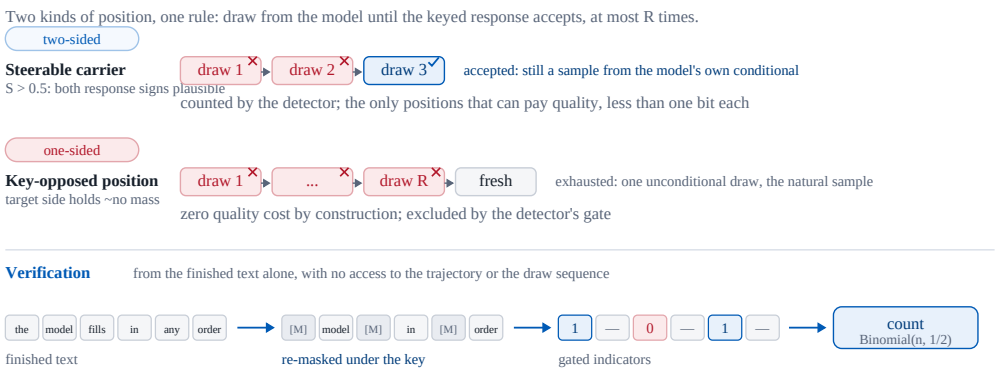

Figure 1: One rule, two regimes. A two-sided carrier accepts within a few redraws — still a sample from the model's conditional, and the only kind of position that pays quality. A key-opposed position rejects every draw and emits the natural sample at zero cost; the gate excludes it. The verifier re-masks under the key and counts gated indicators against an exact binomial null, with no access to the trajectory.

Only two-sided positions are carriers. The live acceptance mass is bimodal (quartiles 0.02/0.60/0.98): at most positions the model's plausible tokens all answer the challenge the same way. Positions are gated by the orientation-symmetric two-sided mass

$$S_i \;=\; 2\min\!\left(q_i^{+}, q_i^{-}\right), \qquad q_i^{\pm} = \Pr_{v\sim p_i^{\text{base}}}\left[\pm g_i(v) > 0\right], \tag{7}$$

computed under the block-masked conditional: a position with $S_i \le 0.5$ is not counted, since if the key opposes it retries cannot help and the fallback makes it a certain miss rather than a coin flip. S_i is invariant under swapping the pair's arms and never sees the key's direction or the message, and it is computed from the block-masked conditional, so the verifier reconstructs the identical carrier set from the finished text. The challenge pair itself is chosen per position among the $\binom{L}{2}$ keyed patterns by the same S_i, which raises the mean two-sided mass from 0.28 (a fixed near/far pair) to 0.45.

5 Why Substitution Cannot Pay: The Measurements Behind the Design

The first version of ReTrace embedded by substitution: at keyed carrier positions, replace the model's token with an admissible alternative whose contrast points the right way, subject to a per-token fluency budget τ (a substitution is admissible iff $p(w)/p(w^\star) \ge e^{-\tau}$). It does not work, and the reason generalises beyond our implementation.

There is nothing to substitute. Across 36 configurations spanning three carrier-selection rules, three budgets and two guidance weights, the median carrier position admits *exactly one* token at $\tau = 3$, i.e. $p(w)/p(w^\star) \ge 0.05$, with no truncation of the candidate slate. The best configuration inside a $1.35\times$ perplexity budget reaches TPR@1% $= 0.10$; matched local dgMARK reaches 0.86 at $1.23\times$.

Better generation makes it worse. Our harness initially ranked positions for low-confidence re-masking by the maximum of the Gumbel-perturbed logits rather than by $p(x_0)$ under the clean softmax. Repairing it to match the reference sampler moved draft perplexity from 22.2 to 9.5 — and halved the denoiser's entropy, from 0.655 to 0.319, shrinking the admissible set by 34–60% ($|A(3)|$: 2.5 → 1.6; $|A(6)|$: 14.1 → 5.6). Part of the early design's apparent strength came from a degraded generator. *Improving a diffusion language model's generation quality reduces its post-hoc watermarkability*, because both quantities are governed by the sharpness of the same conditionals.

Method	Setting	PPL ↓	Ratio ↓	TPR at FPR ↑ 5%	1%	0.1%	z	Det. fwd. ↓
Generation-time watermarks (the watermark is applied while the text is produced)								
No watermark	block dec., top-k 3	9.94	1.00×	–	–	–	+0.04	–
dgMARK (Hong & No, 2026)	$k=1$	15.40	1.55×	0.88	0.74	0.62	+4.35	**0**
dgMARK (Hong & No, 2026)	+3-beam	12.23	1.23×	**0.92**	**0.86**	**0.82**	+5.81	**0**
No watermark	left-to-right	11.61	1.00×	–	–	–	+0.24	–
KGW (Kirchenbauer et al., 2023)	$\delta=1$	11.96	**1.03**×	0.97	**0.93**	0.73	+3.85	**0**
KGW (Kirchenbauer et al., 2023)	$\delta=2$	16.21	1.40×	1.00	1.00	1.00	+6.93	**0**
KGW (Kirchenbauer et al., 2023)	$\delta=3$	14.05	1.21×	1.00	1.00	1.00	+8.92	**0**
Post-hoc substitution — ReTrace V1, archived (§5)								
No watermark	$T=0.8$, full vocab	26.21	1.00×	–	–	–	−0.03	–
ReTrace V1	keyed carrier	27.7	1.11×	0.00	0.00	0.00	+0.73	9
ReTrace V1	best under 1.35× ppl	30.5	1.17×	0.10	0.10	0.00	+0.58	9
ReTrace V1	generation-time, two-phase	203.7	7.77×	0.12	0.00	0.00	+1.14	9
Response-guided resampling — ReTrace V3, presence-only, 512 tok, n=30, matched control								
No watermark	matched scheduler	11.38	1.00×	0.00	0.00	0.00	–	–
ReTrace V3	$R=1$	11.0	**0.97**×	0.57	0.43	0.20	–	144
ReTrace V3	$R=2$	10.6	**0.93**×	0.70	0.57	0.40	–	144
ReTrace V3, held-out prompts (skip 600), n=30, 512 tok; ratios paired against the matched control, valid n=16–17								
ReTrace V3	$R=2$	–	1.14×	0.83	0.73	0.63	–	144
ReTrace V3	$R=4$	–	1.40×	0.90	0.83	0.77	–	144
ReTrace V3	$R=8$	–	1.61×	0.87	**0.87**	**0.80**	–	144

Table 1: Detectability against the quality actually paid. One machine, checkpoint and prompt protocol; each block is scored against a control under its own decoding regime, and *Ratio* is perplexity relative to that control (ReTrace: paired against a matched-scheduler control). Non-watermarked arms sit at $|z| \leq 0.25$. KGW is scored on distinct bigrams — re-scoring repetitions gave a 37% FPR at δ=0 — and its required left-to-right regime itself costs 1.17× against block decoding. *Det. fwd.* counts masked-sequence evaluations per detection.

Moving to generation time does not rescue substitution. A guidance table valid during generation needs every non-carrier token final first, which forces a two-phase schedule that costs 1.98× perplexity *before any watermark* — more than dgMARK's entire cost. Substitution buys evidence with fluency at a rate set by the model's conditionals, and that rate is unfavourable and worsening; a viable embedding must never override the model's choice, which is what §4.3 does.

6 Experiments

One TITAN RTX, `GSAI-ML/LLaDA-8B-Instruct` in fp16; C4 `realnewslike` prompts truncated to 300 characters (dgMARK's protocol); 256–512 tokens at steps = gen_length. Quality is GPT-2-large perplexity against a control *under the same decoding regime* — for ReTrace, the same generator with steering disabled, so the pairing isolates the watermark. Detectability is TPR at fixed FPR, never raw z; development prompts are never reused, and confirmatory runs draw disjoint corpus offsets with per-document keys $K_d = \mathrm{HMAC}(K, \mathrm{nonce}_d)$.

6.1 Main Results and Analysis

The watermark is quality-free, and the earlier evidence against this was a harness artifact. Paired against the matched control, the watermark costs a ΔNLL median of −0.027 at R=1 and −0.037 at R=2 (ratios 0.97×, 0.93×; n=30), replicated on a disjoint held-out set at 0.980× (n=40). Every larger figure previously on record traced to pairing against a differently-scheduled reference generator, and the fingerprint of that confound — R=1 and R=2 showing the same ΔNLL — is what exposed it. The mechanism says this is not luck: at a one-sided position the emitted token is the natural sample whichever way the coin lands, so only genuinely two-sided carriers pay, each less than one bit.

Retries buy detection with arithmetic, not compute — but they do spend the quality budget. On held-out prompts (n=30, 512 tokens), TPR@1% climbs 0.73 → 0.83 → 0.87 across R=2/4/8 while a retry costs no forward pass (Figure 2); the paired quality on the same run is 1.14×, 1.40× and 1.61×. The compute is free; the $-\log q$ budget at two-sided carriers is not, and more retries extract more of it. The operating points that survive comparison are therefore the low-R ones: at

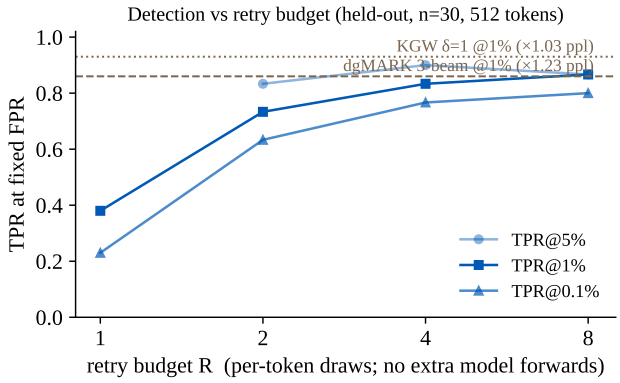

Figure 2: Retries buy detection with arithmetic, but spend the quality budget. n=30 held-out 512-token documents, per-document keys, control at zero false positives; paired quality 1.14× /1.40× /1.61× at R=2/4/8. Reference lines are this paper's local baseline reproductions.

Nominal α	Mean FPR	Worst-key FPR	Verdict
0.10	0.0121	0.0583	valid ($\leq \alpha$)
0.05	0.0037	0.0250	valid ($\leq \alpha$)
0.01	0.0000	0.0000	valid ($\leq \alpha$)

Table 2: False positive control holds per key, worst key included. 20 keys × 120 non-watermarked generations under the deployed rule. Deployment uses one key across documents, so the per-key guarantee is the one that matters; an earlier Gaussian-tail version measured 0.145 at nominal 0.10 and was withdrawn.

R=4 the detection is level with locally reproduced dgMARK 3-beam (0.86) but at higher quality cost (1.40× against 1.23×), so dgMARK dominates that point, whereas nothing in the table reaches the $R\leq$2 region's quality at any detection strength. Length is the other free axis: at R=1, doubling the document from 256 to 512 tokens lifts 1%-TPR from 0.25 to 0.40 purely by growing the presence count, the same length–power trade dgMARK reports.

The simplest detector is the right one. On identical held-out caches the gated count, the gated likelihood ratio and an all-position likelihood ratio are indistinguishable (0.38 at 1% each at R=1), ungated counting is strictly worse (0.33), and every variant holds its null on the control arm. Likelihood weighting buys nothing here because its extra evidence — one-sided positions failing *visibly* — only materialises when the embedder has tried and been refused, which takes retries; at the operating points that matter the deployed detector is the gated indicator with its exact binomial null and no Monte-Carlo machinery.

False positive control holds at the worst key. Twenty keys × 120 non-watermarked generations give 2400 null draws under the deployed decision rule: empirical FPR is at or below nominal at every level *per key*, the worst key included (0.058 at α=0.10, 0.025 at 0.05, 0.000 at 0.01; Table 2), and the matched control arms of every later run sat at zero false positives. An earlier revision reported a Gaussian tail here and measured 0.145 at nominal 0.10; the exact construction is what allows the guarantee to be stated per document rather than on average.

Robustness is the weak axis, and the mechanism says why. Same-model re-denoising of 10% of positions collapses R=1 detection from 0.35 to 0.05 at 1% (deletion likewise), while the attacked control stays at α — the attack erases evidence rather than triggering the detector. The fragility is structural: every block's challenges condition on all earlier blocks, so one edit perturbs the contrast of every later position. Whether the higher rates at R=4–8 buy usable headroom is measured on the saved held-out outputs, and the honest current statement is that token-hash watermarks keep an advantage under heavy post-editing.

Detection cost. Verification evaluates $L+1 = 9$ masked sequences per 32-token block — 144 for a 512-token document, independent of payload size, batchable, roughly ten seconds on this 2018 GPU and sub-second on a modern accelerator. Hash detectors need none. The comparison this prices is a verification API against a mass-scanning filter, and the method is only proposed for the former.

7 Conclusion

A diffusion language model can be applied to a finished text and not only used to produce one, and ReTrace makes that the verification channel: corrupt the text under a secret key, re-denoise, and read whether the model's reconstruction response agrees with a keyed orientation. Embedding never leaves the model's own conditional — a keyed rejection-resampling with a capped retry budget — and against a matched control it is quality-neutral at the low end ($0.93\times$–$0.98\times$ at $R\leq 2$ across independent prompt sets, one set measuring $1.14\times$) and traces a tunable curve above it: TPR@1% of $0.73/0.83/0.87$ at $1.14\times/1.40\times/1.61\times$ for $R{=}2/4/8$ on held-out 512-token documents. The frontier position is honest rather than dominant — locally reproduced dgMARK (0.86 at $1.23\times$) dominates the $R\geq 4$ points, while nothing in the comparison reaches the $R\leq 2$ region's quality. The null is exact per document under the ideal-PRF model, and false positive control holds empirically at the worst of twenty keys.

What separates ReTrace from the watermarks it is compared against is the object verified rather than the strength read. A token-hash detector reads the symbols and is therefore free, fast and robust to the model's own opinion of the text; ReTrace reads the model's response, which costs a fixed number of masked evaluations, and inherits both the good and the bad of context: an exactly reproducible carrier set on one side, and edit damage that propagates through every later block's challenges on the other. Post-editing robustness is accordingly where token-hash schemes keep a real advantage, and hardening the response channel against it — shorter challenge contexts, redundancy across blocks — is the next thing this line of work must address.

References

Scott Aaronson and Hendrik Kirchner. Watermarking gpt outputs, 2023. https://www.scottaaronson.com/talks/watermark.ppt.

Marianne Arriola, Subham Sekhar Sahoo, Aaron Gokaslan, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Justin T. Chiu, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *ICLR*, 2025.

Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *NeurIPS*, 2021.

Arka Bagchi, Akhil Bhimaraju, Moulik Choraria, Daniel Alabi, and Lav R. Varshney. Watermarking discrete diffusion language models. *arXiv preprint arXiv:2511.02083*, 2025.

Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. Accelerated sampling from masked diffusion models via entropy bounded unmasking. In *NeurIPS*, 2025.

Adam Block, Alexander Rakhlin, and Ayush Sekhari. Gaussmark: A practical approach for structural watermarking of language models. In *ICML*, 2025.

Ruibo Chen, Yihan Wu, Yanshuo Chen, Chenxi Liu, Junfeng Guo, and Heng Huang. A watermark for order-agnostic language models. In *ICLR*, 2025.

Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. In *COLT*, 2024.

Jiayi Fu, Xuandong Zhao, Ruihan Yang, Yuansen Zhang, Jiangjie Chen, and Yanghua Xiao. Gumbelsoft: Diversified language model watermarking via the gumbelmax-trick. In *ACL*, 2024.

Thibaud Gloaguen, Robin Staab, Nikola Jovanović, and Martin Vechev. Watermarking diffusion language models. In *ICLR*, 2026. arXiv:2509.24368.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.

Pyo Min Hong and Albert No. dgMARK: Decoding-Guided Watermarking for Diffusion Language Models. In *ICML*, 2026. arXiv:2601.22985.

Nikola Jovanović, Robin Staab, and Martin Vechev. Watermark stealing in large language models. In *ICML*, 2024.

Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham M. Kakade, and Sitan Chen. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *ICML*, 2025.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *ICML*, 2023.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Manli Shu, Khalid Saifullah, Kezhi Kong, Kasun Fernando, Aniruddha Saha, Micah Goldblum, and Tom Goldstein. On the reliability of watermarks for large language models. In *ICLR*, 2024.

Kalpesh Krishna, Yixiao Song, Marzena Karpinska, John F. Wieting, and Mohit Iyyer. Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense. In *NeurIPS*, 2023.

Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. Robust distortion-free watermarks for language models. *TMLR*, 2024.

Inception Labs, Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sarah Birnbaum, Ziyang Luo, Yanis Miraoui, and Akash Palrecha. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025.

Yuqing Liang, Jiancheng Xiao, Wensheng Gan, and Philip S. Yu. Watermarking techniques for large language models: A survey. *Artificial Intelligence Review*, 2026.

Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *ICML*, 2024.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *NeurIPS*, 2025.

Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *ICLR*, 2025.

Fabien A. P. Petitcolas, Ross J. Anderson, and Markus G. Kuhn. Information hiding — a survey. In *Proceedings of the IEEE*, 1999.

Ohad Raban, Ethan Fetaya, and Gal Chechik. LR-DWM: Efficient watermarking for diffusion language models. *arXiv preprint arXiv:2601.12376*, 2026.

Subham Sekhar Sahoo, Marianne Arriola, Aaron Gokaslan, Edgar Marroquin Marroquin, Alexander M. Rush, Yair Schiff, Justin T. Chiu, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *NeurIPS*, 2024.

Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. In *NeurIPS*, 2024.

Qingyan Wei, Yaojie Zhang, Zhiyuan Liu, Pengfei Zeng, Yu Wang, Biao Qi, Dong Liu, and Linfeng Zhang. Accelerating diffusion large language models with slowfast sampling: The three golden principles. In *ICLR*, 2026.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. In *ICLR*, 2026.

Linyu Wu, Linhao Zhong, Wenjie Qu, Yifan Li, Yue Liu, Shengfang Zhai, Chunhua Shen, and Jiaheng Zhang. DMark: Order-agnostic watermarking for diffusion large language models. *arXiv preprint arXiv:2510.02902*, 2025.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

Xuandong Zhao, Yu-Xiang Wang, and Lei Li. Protecting language generation models via invisible watermarking. In *ICML*, 2023.

Xuandong Zhao, Prabhanjan V. Ananth, Lei Li, and Yu-Xiang Wang. Provable robust watermarking for ai-generated text. In *ICLR*, 2024.

Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025.